More Scheme Lists, Scheme Eval / Apply

Computer Science Mentors 61A

April 14, 2026 – April 17, 2026

1 Scheme Lists

1. Identify the bug(s) in this program.

```
> (sum-every-other '(1 2 3))
4
> (sum-every-other '())
0
> (sum-every-other '(1 2 3 4))
4
> (sum-every-other '(1 2 3 4 5))
9

(define (sum-every-other lst)
    (cond ((null? lst) lst)
          (else (+ (cdr lst)
                   (sum-every-other (car lst))))))
```

Created by Anaisha Das and Tavisha Patel. Worksheet feedback: forms.gle/q67BZrHHfoKbF6V28

2. Implement `waldo`. `waldo` returns `#t` if the symbol waldo is in a list.

```
scm> (waldo '(1 4 waldo))
#t
scm> (waldo '())
#f
scm> (waldo '(1 4 9))
#f

(define (waldo lst))
```

3. **Extra challenge:** Define `waldo` so that it returns the index of the list where the symbol waldo was found (if waldo is not in the list, return `#f`).

```
scm> (waldo '(1 4 waldo))
2
scm> (waldo '())
#f
scm> (waldo '(1 4 9))
#f

(define (waldo lst)









)
```

4. (a) Define `append`, which takes in two lists and returns a new list with all the elements of the first list followed by all the elements of the second. Do not use the built-in `append` function.

```
> (append '(1 2 3) '(4 5 6))
(1 2 3 4 5 6)

(define (append lst1 lst2)









)
```

   (b) Define `reverse`. Hint: use `append`.

```
> (reverse '(1 2 3))
(3 2 1)

(define (reverse lst)









)
```

2 Scheme Eval/Apply and Programs as Data

1. What will Scheme output?

```
scm> (define c 4)

scm> ((define (x) 1))

scm> (x)

scm> ((lambda (x y) (+ c)) 1 2)

scm> (eval 'c)

scm> '(cons 1 nil)

scm> (eval (list 'if '(even? c) 1 2))

scm> (let ((a (+ 3 1)) (b 3)) (+ a b) (/ a b))
```

2. Circle or write the number of calls to `scheme_eval` and `scheme_apply` for the code below.

```
(if 1 (+ 2 3) (/ 1 0))
```

scheme_eval	1	3	4	6
scheme_apply	1	2	3	4

```
(or #f (and (+ 1 2) 'apple) (- 5 2))
```

scheme_eval	6	8	9	10
scheme_apply	1	2	3	4

```
(define (square x) (* x x))

(+ (square 3) (- 3 2))
```

scheme_eval	2	5	14	24
scheme_apply	1	2	3	4

```
(define (add x y) (+ x y))

(add (- 5 3) (or 0 2))
```

3 Optional

1. Write a function plan-coffee-tour that takes two lists of Berkeley coffee shops and creates an optimized tour according to the following rules: The function creates a tour by alternating shops from each list (similar to interleaving) If a coffee shop appears in both lists, it should only be visited once in the tour at its first occurrence If one list is longer than the other, the remaining unique shops should be added to the end of the tour

```
scm> (plan-coffee-tour '(binge strada philz) '(philz blue-bottle
   binge))
(binge philz strada blue-bottle)

scm> (plan-coffee-tour '(strada mind peets) '(elaichi-co philz))
(strada elaichi-co mind philz peets)

scm> (plan-coffee-tour '(strada qargo) '(strada qargo peets))
(strada qargo peets)

scm> (plan-coffee-tour '() '(delah signal))
(delah signal)

(define (plan-coffee-tour lst1 lst2)
    (cond ((_____________________________) lst2)
        ((_____________________________) lst1)
        (else
        (let ((first (car lst1))
                (rest1 (cdr lst1))
                (rest2 (_____________________________)))
            (if (_____________________________)
                (cons first (plan-coffee-tour rest1 rest2))
                (cons first (cons (_____________________________)
                                  (plan-coffee-tour rest1
                                     (_____________________________)))))))))
```